

Ruta técnica para crear tu propio scanner de surebets gratis

Veredicto

Si tu universo objetivo es BetPlay ¹, RushBet ², Stake ³, Codere ⁴, Betano ⁵, Wplay ⁶ y, si por “betson” te referías a Betsson ⁷, la ruta más realista para algo **preciso, gratis y adaptado a tus cuentas reales** no es comprar una API comercial; es construir un **colector propio**. El motivo es simple: el plan gratis de BetBurger deja ver surebets con límites, su API requiere suscripción, y las APIs genéricas de cuotas que sí ofrecen free tier o prueba siguen teniendo límites de uso o modelos comerciales que no encajan bien con tu restricción de “todo gratis”. ⁸

Mi recomendación concreta es esta: **hazlo en Python**, con un backend local que abra sesión en cada casa, capture sus cuotas, las normalice, calcule arbitrajes y te mande alertas. No arrancaría por live ni por apuesta automática. Arrancaría por **prematch** y en modo **alert-only**. BetBurger mismo describe el prematch como el modo más lento y más amigable para principiantes, mientras que en live la latencia manda y las oportunidades duran segundos; además, en su propia guía de API reconoce que mucha gente usa estos flujos sólo para alertas antes de automatizar más. ⁹

Cómo funciona BetBurger ¹⁰

Según su propia documentación, BetBurger funciona como un escáner que **recoge cuotas, analiza si hay arbitraje y muestra inmediatamente** la lista de oportunidades. En sus páginas públicas describe un servicio que recolecta odds de muchas líneas de apuesta y luego analiza esos datos para detectar arbs, surebets, middles y valuebets. ¹¹

La sección de surebets de BetBurger no es sólo “una lista de partidos”. Su guía de uso describe cuatro piezas principales en la vista: **lista de arbs, dashboard, calculadora y lista adicional del mismo evento**. El dashboard permite ordenar por porcentaje, antigüedad, hora de inicio y ROI; también soporta auto-actualización, alertas sonoras o pop-up y agrupación por evento. La calculadora sirve para repartir stakes, redondear importes, considerar fees y abrir odds adyacentes cuando una pata ya cambió. ¹²

Otra parte clave de la sección surebets son los **filtros**. BetBurger documenta filtros por casas “requeridas”, países, tipos de outcome y periodos live. Eso significa que su producto no sólo “encuentra surebets”: también reduce ruido y deja ver únicamente las combinaciones que te interesan operar. En otras palabras, su valor no está sólo en la matemática, sino en la **selección, la normalización y el filtrado operativo**. ¹³

Hay además un concepto muy importante para tu clon: **clones y espejos**. BetBurger explica que los clones son casas con la misma línea y que, aunque a veces cambie el diseño o el margen, alrededor del 90% de mercados/eventos pueden coincidir. También documenta espejos o dominios alternativos. Esto importa

muchísimo porque te dice cómo pensar tu arquitectura: no debes modelar todo “por dominio web”, sino por **familia de feed** y por **variante regional/dominio**. ¹⁴

Eso no es teoría: BetBurger ha ido publicando cambios concretos que muestran que algunas marcas regionales deben tratarse aparte. El **17 de septiembre de 2025** informó que empezó a recibir datos directos de Rushbet y BetPlay como casas separadas; el **21 de abril de 2026** dijo que Wplay pasó a tratarse como bookmaker principal con datos directos; y también publicó que la versión colombiana de Stake debía manejarse separadamente de la internacional porque las cuotas diferían. La lectura técnica es clarísima: en tu sistema necesitas **adaptadores propios por fuente regional real**, aunque varias marcas parezcan “la misma casa” por fuera. ¹⁵

Además, la lista pública de BetBurger sí incluye justamente las referencias que te interesan: aparecen Rushbet, Betplay, Wplay y variantes como BetanoCO, CodereCO, BetssonCO y StakeCO. O sea, el objetivo que tienes en mente es técnicamente el mismo tipo de cobertura que ellos manejan en su scanner. ¹⁶

Sobre el modelo de negocio, BetBurger ofrece un plan gratis con límites: su FAQ menciona surebets live de hasta 1% y valuebets de hasta 2%, y sus páginas comerciales reiteran que en modo gratuito sólo se muestran surebets hasta 1%. Su API, en cambio, es de pago y la integración pasa por filtros, token privado y endpoints separados para live y prematch. Eso encaja exactamente con tu motivación: quieres construir lo que BetBurger ya hace, pero sin depender de esos límites. ¹⁷

Qué tendrías que construir

Lo que te toca construir no es una sola app, sino una **cadena de cinco capas** bien separadas.

La primera capa es **captura**. Necesitas un adaptador por casa de apuestas que sepa entrar, navegar a los deportes y mercados que te interesan, y leer la fuente real de cuotas. En algunos casos esa fuente será una respuesta JSON/XHR detrás de la página; en otros, tocará leer el DOM final. La idea correcta es que cada casa sea un módulo independiente, para que cuando una cambie no se te rompa todo el sistema. Esta separación por fuente/casa es coherente con cómo BetBurger organiza bookies, required bookies, clones y estados de escaneo. ¹⁸

La segunda capa es **normalización**. Todas las casas nombran distinto el mismo partido, el mismo mercado y la misma selección. Tú necesitas un modelo canónico con campos como: `bookmaker`, `sport`, `league`, `event_start`, `home`, `away`, `market_type`, `period`, `line`, `selection`, `odds_decimal`, `url`, `last_seen_at`. Ese modelo es el corazón del proyecto. Sin él, no hay comparación fiable.

La tercera capa es **matching**. Aquí unes “Real Madrid vs Barcelona” con “Barcelona @ Real Madrid”, o “Over 2.5” con “Más de 2.5”. También decides si “moneyline”, “1X2”, “hándicap asiático” y “total” pertenecen al mismo grupo lógico o no. Este problema es más difícil que la fórmula matemática. De hecho, BetBurger en su propia documentación sobre APIs insiste en que la calidad real del producto está en el matching correcto entre bookies y en filtrar surebets obsoletas o mal emparejadas. ¹⁹

La cuarta capa es el **motor de arbitraje**. En términos matemáticos, una surebet existe cuando la suma de inversos de las cuotas que cubren todos los resultados excluyentes es menor que 1:

$$s = \sum_i \frac{1}{o_i} < 1$$

Si tienes un bankroll total B , el stake de cada pata puede calcularse como:

$$stake_i = \frac{B}{s \cdot o_i}$$

y el ROI garantizado como:

$$ROI = \frac{1}{s} - 1$$

Eso sirve para 2-way y 3-way. La parte difícil no es la fórmula; es asegurarte de que las cuotas realmente cubren **el mismo evento, el mismo periodo y la misma línea**.

La quinta capa es **alerta y operación**. Ahí decides si sólo quieres avisos por pantalla, una API interna o un bot por Telegram ²⁰. Mi consejo es que tu MVP no coloque apuestas: que sólo muestre la oportunidad, el reparto de stakes, la antigüedad de cada cuota y los enlaces de apertura a cada casa. Ese patrón también aparece en la forma en la que BetBurger presenta su calculadora, sus enlaces y sus flujos de API para alertas/manualidad antes de automatizar más. ²¹

Tecnologías gratis que usaría

Yo lo haría en **Python**. No porque sea la única opción, sino porque para este caso mezcla muy bien automatización de navegador, concurrencia, tipado, persistencia local y cálculo. La pareja clave sería **Playwright + asyncio + FastAPI + SQLite**. Playwright tiene soporte oficial para interceptar tráfico de red, reutilizar estado autenticado y trabajar con contextos aislados; `asyncio` está pensado para concurrencia I/O-bound; FastAPI te da una API moderna y tipada; y SQLite te deja empezar sin servidor, sin configuración y sin costo. ²²

La mejor combinación, para un proyecto como el tuyo, sería esta:

- **Playwright** para login, navegación, captura de XHR/fetch y apertura de enlaces.
- **asyncio** para correr varios colectores a la vez sin bloquear.
- **FastAPI** para exponer una API interna y una UI mínima local.
- **SQLite** al inicio; más adelante, si realmente escalas a procesos múltiples o despliegue remoto, migras a PostgreSQL.
- **APScheduler** o cron para ejecutar rondas de escaneo a intervalos controlados.
- **Telegram Bot API** para alertas inmediatas. ²³

No complicaría el front al inicio. Haría una interfaz web local muy simple que muestre: evento, mercado, casas, cuotas, edad de la oportunidad, ROI, stakes y botones para abrir cada bookie. Lo importante no es que “se vea lindo”; es que **sea confiable y rápido**.

Cómo hacerlo preciso y efectivo

La precisión empieza por la autenticación. Lo correcto es iniciar sesión manualmente una vez por casa y guardar el estado autenticado para reutilizarlo. Playwright documenta precisamente este flujo con `storage_state`, y además advierte que esos archivos contienen cookies y datos sensibles y no deben subirse al repositorio. En tu caso, eso te permite tener una sesión persistente por casa y evitar logins repetidos en cada corrida. ²⁴

La siguiente regla de oro es esta: **no raspes el HTML si puedes leer el JSON que la propia web ya consume**. Playwright documenta monitoreo e interceptación de red para XHR/fetch/HTTP, y la propia explicación comercial de BetBurger sobre APIs insiste en que el “site parsing” puro es frágil frente a cambios de layout, bloqueo y carga dinámica por JavaScript. Tu meta, entonces, no es copiar visualmente la página, sino descubrir el feed interno más estable de cada casa. Además, para pruebas, vale mucho guardar respuestas crudas o HARs de ejemplo para testear normalizadores y matchers sin volver a pegarle a los sitios. ²⁵

También necesitas una política seria de matching. Yo usaría tres niveles: primero, reglas exactas por mercado/periodo/línea; segundo, un diccionario manual de alias de equipos y ligas; tercero, matching difuso con tolerancia pequeña de hora de inicio. En la práctica, si no tienes confianza alta en el matching, no calculas surebet. Mejor perder una oportunidad que alertar una falsa.

Para que el sistema sea realmente efectivo, le pondría **revalidación**. Si aparece una surebet, vuelves a consultar ambas patas justo antes de alertar o de mostrarla como “lista para ejecutar”. BetBurger mismo muestra que el dashboard ordena por antigüedad y que su producto necesita mecanismos para monitorear estado de bookies y posibles fallos; ese es el tipo de disciplina operativa que debes copiar. En tu MVP, además, yo descartaría cualquier oportunidad cuya edad supere un umbral corto y llevaría una métrica de falsos positivos. ²⁶

En cuanto a alcance, empezaría sólo con **mercados simples y muy comparables**: `1X2`, `moneyline`, `totales` y, si todo va bien, `hándicaps`. Esa misma taxonomía de outcomes aparece en la documentación de filtros de BetBurger y es la base natural para un comparador robusto. No arrancaría con props, corners, bookings o specials, porque disparan la complejidad del matching. ²⁷

Por eso mi recomendación operativa sigue siendo la misma: **prematch primero, live después**. BetBurger distingue claramente ambos modos y subraya que el prematch cambia más lento, mientras live exige mucha más velocidad. La arquitectura final la puedes diseñar desde ya para soportar live, pero el primer resultado útil y realista te lo va a dar un scanner prematch. ²⁸

Riesgos, límites y orden de ejecución

La limitación real de este proyecto no es la matemática: es el **mantenimiento**. Incluso BetBurger, cuando habla de APIs, admite que parsear casas en tiempo real exige mantenimiento continuo y que el gran problema del scraping es romperse cuando cambia el sitio. Por eso la arquitectura modular no es un lujo; es la única forma sana de hacerlo. ¹⁹

Hay otro límite práctico: aunque el arbing no sea una operación ilegal por sí misma, los operadores pueden no ser amigables con esa práctica. BetBurger lo dice sin rodeos: considera la surebet legal, pero reconoce que desde el punto de vista del bookmaker puede violar sus reglas, y también explica que una sanción común es la limitación de cuenta. Eso no te impide construir el scanner, pero sí significa que tu sistema debe estar pensado primero para **detectar y avisar**, no para empujarte ciegamente a operarlo todo. ²⁹

El orden correcto de ejecución, si quieres llegar a algo parecido a BetBurger sin pagar, sería este. Primero, un MVP prematch con **tres casas y dos o tres mercados**; yo empezaría por BetPlay, RushBet y Wplay, porque BetBurger ha mostrado recientemente que esas fuentes regionales merecen tratamiento directo propio. Después añadiría Betano y Codere. Luego sumaría StakeCO y BetssonCO. Sólo cuando el matching y la refrescancia estén sólidos metería live. ³⁰

En una frase: **sí puedes hacerlo, y la mejor opción para ti es Python + Playwright + asyncio + FastAPI + SQLite + alertas por Telegram, corriendo localmente, con arquitectura por adaptadores y enfoque prematch-first**. Ese es, hoy, el camino más preciso, más efectivo y más compatible con tu requisito de que todo sea gratis. ³¹

¹ ⁴ ¹³ ¹⁸ Two Lists of Bookmakers: All bookmakers and Required | BetBurger
<https://www.betburger.com/manual/bookmaker-lists>

² ³⁰ <https://www.betburger.com/news/wplay-separate-bookmaker>
<https://www.betburger.com/news/wplay-separate-bookmaker>

³ ²² ²³ ²⁵ ³¹ Network
https://playwright.dev/docs/network?utm_source=chatgpt.com

⁵ ¹² ²⁰ ²⁶ BetBurger Surebet Service Guide
<https://www.betburger.com/blog/our-service-user-guide>

⁶ ¹⁵ <https://www.betburger.com/news/rushbet-betplay-are-available-as-separate-bookmakers>
<https://www.betburger.com/news/rushbet-betplay-are-available-as-separate-bookmakers>

⁷ ¹⁶ <https://www.betburger.com/es/bookmakers>
<https://www.betburger.com/es/bookmakers>

⁸ ¹⁷ <https://www.betburger.com/faq/400>
<https://www.betburger.com/faq/400>

⁹ ²⁸ ²⁹ ¹ What is Surebet? | Formula with Example
<https://www.betburger.com/newbies/what-is-surebet>

¹⁰ ²¹ Surebet Calculator for sports arbitrage betting – FREE for 2, 3, 4 way
<https://www.betburger.com/surebet-calculator>

¹¹ <https://www.betburger.com/about-us>
<https://www.betburger.com/about-us>

¹⁴ <https://www.betburger.com/es/manual/bookmaker-clones-and-mirrors>
<https://www.betburger.com/es/manual/bookmaker-clones-and-mirrors>

¹⁹ <https://www.betburger.com/blog/what-is-odds-api-in-betting>
<https://www.betburger.com/blog/what-is-odds-api-in-betting>

24 Authentication | Playwright Python

https://playwright.dev/python/docs/auth?utm_source=chatgpt.com

27 Outcome filtering | BetBurger

<https://www.betburger.com/manual/filtering-the-outcomes>